

Nested Queries

Simple Nested Queries

Use of Set Operations (Union , Intersect & Except)

IN Operator

- The IN operator allows you to specify multiple values in a WHERE clause.
- The IN operator is a shorthand for multiple OR conditions

```
SELECT CNIC, Name , GPA
```

```
FROM Student
```

```
WHERE Name IN( 'Maria' , 'Maida','Madiha');
```

Student

CNIC	Name	Address	GPA	Did
------	------	---------	-----	-----

IN Operator – Same Queries.

```
SELECT CNIC, Name , GPA
```

```
FROM Student
```

```
WHERE Name IN( 'Maria' , 'Maida','Madiha');
```

```
SELECT CNIC, Name , GPA
```

```
FROM Student
```

```
WHERE Name = 'Maria' or Name = 'Maida' or Name = 'Madiha';
```

Student

CNIC	Name	Address	GPA	Did
------	------	---------	-----	-----

Example

- Show employees names who are working on project number 1 or 2 or 3 or 10.
- = OR

Select fname

From employee join works_on on essn = ssn

Where pno in (1,2,3,10)

Sub query / Nested Query / Inner Query

- A Nested query is a query within another SQL query and is embedded within the WHERE clause.
-
- Select
 - From
 - Where attribute IN (Select
 From
 Where
)

Sub query / Nested Query / Inner Query

- A subquery is used to return data that will be used in the main query as a condition to further restrict the data to be retrieved.
- Select
- From
- Where attribute IN (
 - Select
 - From
 - Where)

Example

- Show all employees who are working in research department.
- Select *
- From employee
- Where dno in (select dnumber
 From department
 Where dname = research);

Example

- Show all employees who are working in a project
- Select *
- From employee
- Where ssn in (select essn
 From works_on);

Example

- Show all employees who are working in a project located in stafford.
- Select *
- From employee
- Where ssn in (
 select essn
 From works_on
 Where pno in (
 select Pnumber
 From project
 Where location = 'stafford')));

Key word NOT IN

- We can use NOT IN in similar manner as we used IN.
- Show employees who are not managers.
- Select *
- From employee
- Where ssn not in (select mgrssn
From department
);

Set Operation

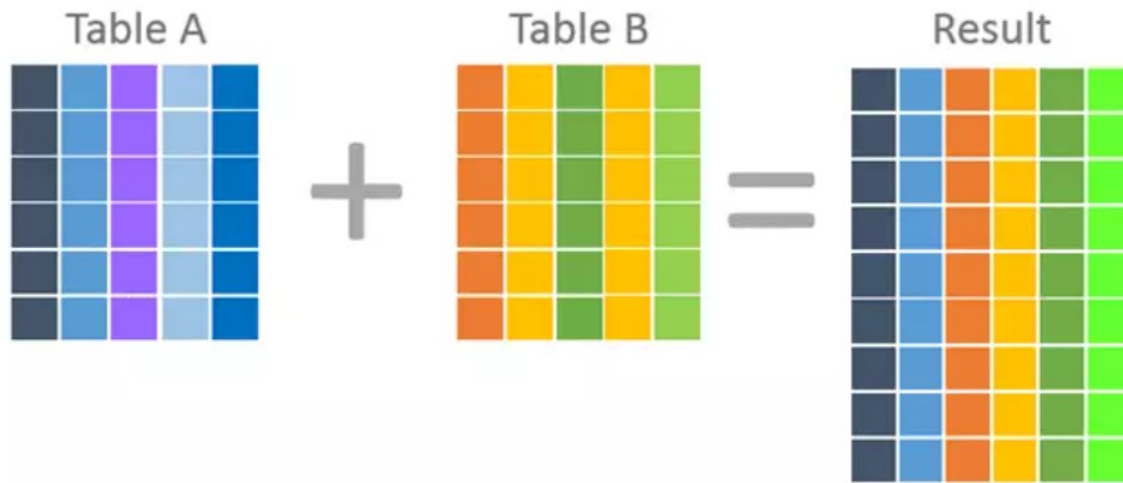
Set Operation

- Union [UNION]
 - Union All [UNION ALL]
 - Intersection [IN]
 - Difference [NOT IN]
-
- $A(1, 2, 3, 6)$ $B(1, 3, 4, 5)$
 - $A \cup B(1, 2, 3, 4, 5, 6)$
 - $A \cap B = (1, 3)$
 - $A - B = (2, 6)$ $B - A = (4, 5)$
 - $A \cup B = B \cup A$
 - $A \cap B = B \cap A$
 - $A - B \neq B - A$

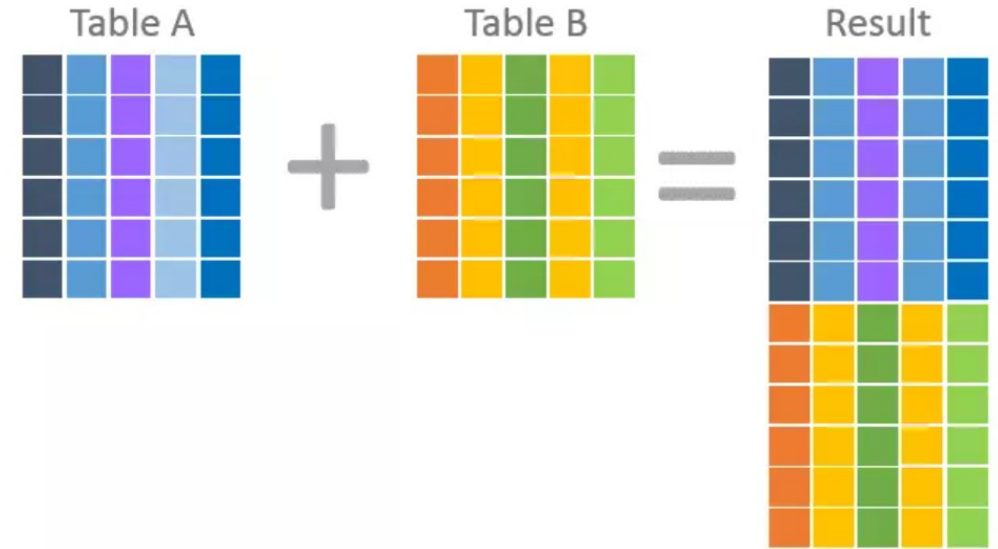
Union

- Sql gives us a keyword of union to achieve the desired results.

Set operation or join



Joins Combine Columns



Unions Combine Rows

<https://www.essentialsql.com/what-is-the-difference-between-a-join-and-a-union/>

Set Operation

Student

CNIC	Name	Address	GPA	Did
------	------	---------	-----	-----

Teacher

CNIC	Name	Address	Salary	Did
------	------	---------	--------	-----

Department

ID	Name	Location
----	------	----------

Name All people (Student + Teacher) From Lahore

```
SELECT CNIC, Name
```

```
FROM Student
```

```
where Address LIKE '%Lahore%'
```

UNION

```
SELECT CNIC , Name
```

```
FROM Teacher
```

```
where Address LIKE '%Lahore%';
```

Name all Students who are also teaching.

```
SELECT Name
```

```
FROM Student
```

```
WHERE CNIC IN (SELECT CNIC  
FROM Teacher);
```

Name all students who are not teaching.

```
SELECT Name
```

```
FROM Student
```

```
WHERE CNIC NOT IN (SELECT CNIC  
FROM Teacher);
```

Union Compatibility

- Two tables must have equal number of attributes.
- Each attribute must have same domain.

Name All people (Student + Teacher) From Lahore

```
SELECT CNIC, Name  
FROM Student  
where Address LIKE '%Lahore%'  
UNION  
SELECT CNIC , Name  
FROM Teacher  
where Address LIKE '%Lahore%';
```


Use of 'Union' or 'OR'

Student

CNIC	Name	Address	GPA	Did
------	------	---------	-----	-----

Teacher

CNIC	Name	Address	Salary	Did
------	------	---------	--------	-----

Department

ID	Name	Location
----	------	----------

Name the student from Department 3 or GPA >3.

Select Name
from Student
Where did = 3 OR GPA >3;

Name Student or Teachers from Lahore.

```
SELECT Name  
FROM Student  
where Address LIKE '%Lahore%'  
UNION  
SELECT Name  
FROM Teacher  
where Address LIKE '%Lahore%';
```

Summary

- You can use Join or Sub query (IN Keyword) interchangeably. Both have same results. And both depicts the Set operation INTERSECTION.
- Union can be achieved with UNION keyword only.
- To have the functionality of minus or Except we need to use NOT IN.

Practice Questions

1. Show department names in which some workers works with salary > 30,000 or it is located in Stafford.
2. Show employee ssn's who are managers but don't have any dependents.
3. Show manager name who have no dependents.
4. Make a list of all project numbers for projects that involve an employee whose last name is smith either as a worker or as a manager of dept who controls the project.
5. List names of employees who do not work on any project.
6. Find the names of all employees who work on at least one project located in Houston but whose department has not located in Houston.

Practice Questions

1. Show employee names who are managers and they have at least one dependent.
2. . Show employee name who are managers but they do not have dependents
3. Show employee name whose superssn is same as superssn of either "javed" or "ahmad".
4. Show project name on which Aliya is not working.
5. Show all dependent name of employees working in IT department.
6. Show employee names who work in research department.
7. Show project names in which an employee 'John Smith' works.
8. Show all project names which are controlled by a department in which 'John Smith' works.
9. Show employee names who has a supervisor with the same salary as themselves.
10. Show employee names who do not work on any project.
11. Show department names which are not controlling any project and has a location in Houston

Show employee names who are managers and they have at least one dependent. nesting

- Select fname
- From employee
- Where ssn in (select mgrssn from department)

And

ssn in (select essn from dependent) ;

Show employee names who are managers and they have at least one dependent. nesting

- Select fname
- From employee
- Where **ssn in (select mgrssn**
 - From department
 - Where mgrssn in (select essn from dependent));

Show employee names who are managers and they have at least one dependent. nesting

- Select fname
- From employee join department on mgrssn =ssn
- Where **ssn in (select essn from dependent);**

Show employee name who are managers but they do not have dependents

- Select fname
- From employee
- Where `ssn in (select mgrssn from department)`

And

`ssn Not in (select essn from dependent) ;`

Co-related queries

Use of Exists and Not Exists

Exists & Not Exists Operator

The EXISTS operator is used to test for the existence of any record in a subquery.

The EXISTS operator returns true if the subquery returns one or more records.

The NOT EXISTS operator returns true if the subquery returns no records at all.

Exists,Not Exists VS IN,NOT IN Operator

IN	Exists
We can use it on subqueries as well as with values.	We can use it only on subqueries.
It executes faster when the subquery result is less.	It executes faster when the subquery result is large. It is more efficient than IN because it processes Boolean values rather than values itself.
Syntax to use IN clause: SELECT col_names FROM tab_name WHERE col_name IN (subquery);	Syntax to use EXISTS clause: SELECT col_names FROM tab_name WHERE [NOT] EXISTS (subquery);

Correlated query

A correlated subquery is a subquery that uses values from the outer query.

Show the detail of employees who is working on at least one project

```
SELECT *  
FROM EMPLOYEE  
WHERE ssn IN  
    ( SELECT DISTINCT Essn  
      FROM WORKS_ON);
```

```
SELECT *  
FROM EMPLOYEE  
WHERE EXISTS  
(  
  SELECT *  
  FROM WORKS_ON  
  WHERE WORKS_ON.Essn =  
    EMPLOYEE.ssn  
);
```

Correlated query

Show employees who do not have any dependents.

```
SELECT *  
FROM EMPLOYEE  
WHERE ssn NOT IN  
  ( SELECT DISTINCT Essn  
    FROM DEPENDENT  
  );
```

```
SELECT *  
FROM EMPLOYEE  
WHERE NOT EXISTS  
(  
  SELECT *  
  FROM DEPENDENT  
  WHERE DEPENDENT. Essn  
    =EMPLOYEE.ssn  
)
```

Examples:

Consider the following two relation "Customers" and "Orders".

Customers

customer_id	lname	fname	website
401	Singh	Dolly	abc.com
402	Chauhan	Anuj	def.com
403	Kumar	Niteesh	ghi.com
404	Gupta	Shubham	jkl.com
405	Walecha	Divya	abc.com
406	Jain	Sandeep	jkl.com
407	Mehta	Rajiv	abc.com
408	Mehra	Anand	abc.com

Orders

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
3	408	2017-01-18
4	404	2017-02-05

1. Using EXISTS condition with SELECT statement

To fetch the first and last name of the customers who placed atleast one order.

```
SELECT fname, lname  
FROM Customers  
WHERE EXISTS (SELECT *  
              FROM Orders  
              WHERE Customers.customer_id = Orders.c_id);
```

Output:

fname	lname
Shubham	Gupta
Divya	Walecha
Rajiv	Mehta
Anand	Mehra

3. Using EXISTS condition with DELETE statement

Delete the record of all the customer from Order Table whose last name is 'Mehra'.

```
DELETE
FROM Orders
WHERE EXISTS (SELECT *
              FROM customers
              WHERE Customers.customer_id = Orders.cid
              AND Customers.lname = 'Mehra');
```

```
SELECT * FROM Orders;
```

Output:

order_id	c_id	order_date
1	407	2017-03-03
2	405	2017-03-05
4	404	2017-02-05

4. Using EXISTS condition with UPDATE statement

Update the lname as 'Kumari' of customer in Customer Table whose customer_id is 401.

```
UPDATE Customers
SET lname = 'Kumari'
WHERE EXISTS (SELECT *
              FROM Customers
              WHERE customer_id = 401);
```


Example

Show the name of all customers who placed at least one order.

Can we do it by nesting?

Customer :

ID	Name	Address	Phone
----	------	---------	-------

Order:

ID	C_id	Delievery_Date	Status
----	------	----------------	--------

Example

Show the name of all customers who placed at least one order.

Customer :

<u>ID</u>	Name	Address	Phone
-----------	------	---------	-------

Order:

<u>ID</u>	C_id	Delievery_Date	Status
-----------	------	----------------	--------

```
SELECT Name
FROM Customer
WHERE EXISTS (SELECT *
              FROM Order
              WHERE Customer.ID = Order.C_id);
```

Example

Show the name of all customers who has not placed any order.

Customer :

<u>ID</u>	Name	Address	Phone
-----------	------	---------	-------

Order:

<u>ID</u>	C_id	Delievery_Date	Status
-----------	------	----------------	--------

```
SELECT Name
FROM Customer
WHERE NOT EXISTS (SELECT *
                   FROM Order
                   WHERE Customer.ID = Order.C_id);
```

Try these with Correlated Method.

1. Show employee names who are managers and they have at least one dependent.
2. Show employee name who are managers but they do not have dependents.
3. Show project name on which Aliya is not working.
4. Show all dependent name of employees working in IT department.
5. Show employee names who work in research department.
6. Show project names in which an employee 'John Smith' works.
7. Show all project names which are controlled by a department in which 'John Smith' works.
8. Show employee names who has a supervisor with the same salary as themselves.
9. Show employee names who do not work on any project.
10. Show department names which are not controlling any project and has a location in Houston.

Query 1

Select the project names of projects that have employee with last name smith involved as manager or as a worker.

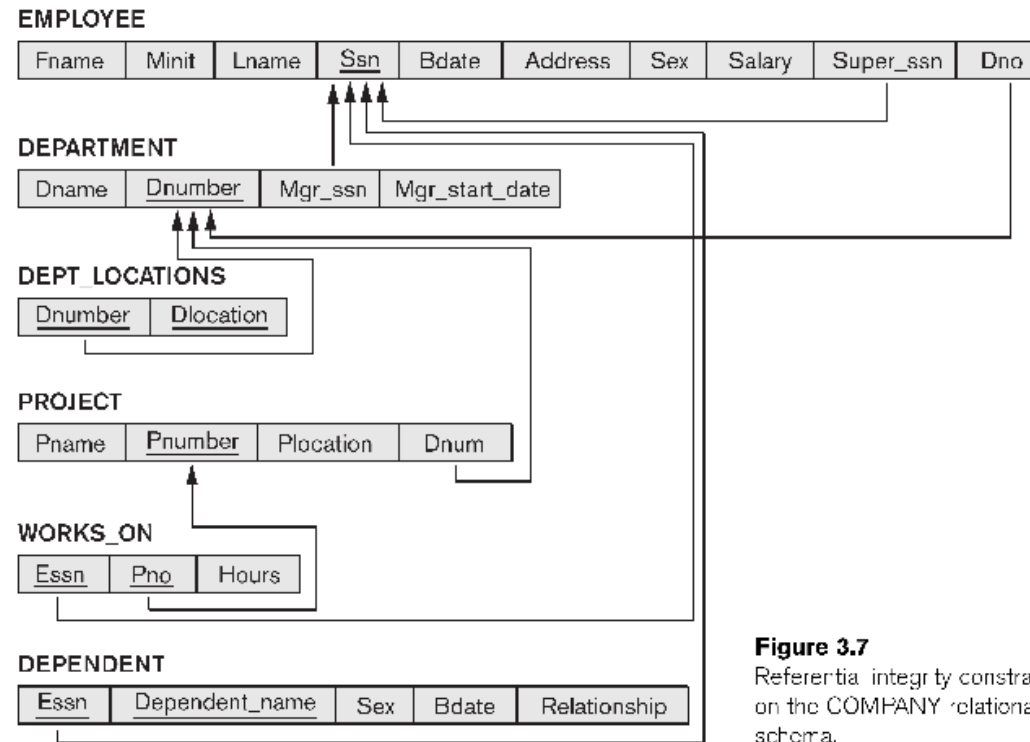


Figure 3.7

Referential integrity constraints displayed on the COMPANY relational database schema.

Query 1

Select the project names of projects that have employee with last name smith involved as manager or as a worker.

- Step one: Break query into pieces.

Project number of all those project in which smith is involved as worker

Project number of all those projects in which smith is involved as manager

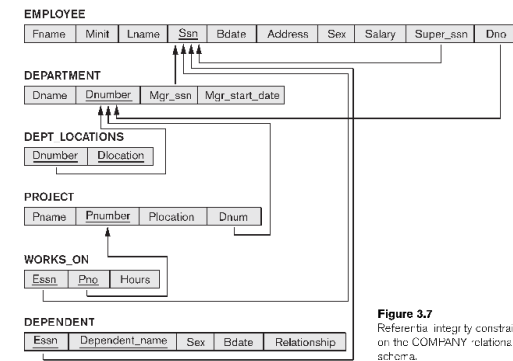


Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

Query 1

Project number of all those project in which smith is involved as worker

Select wo.pno

From WORKS_ON AS wo INNER JOIN EMPLOYEE AS
e ON e.ssn = wo.essn

WHERE e.Lname = 'smith';

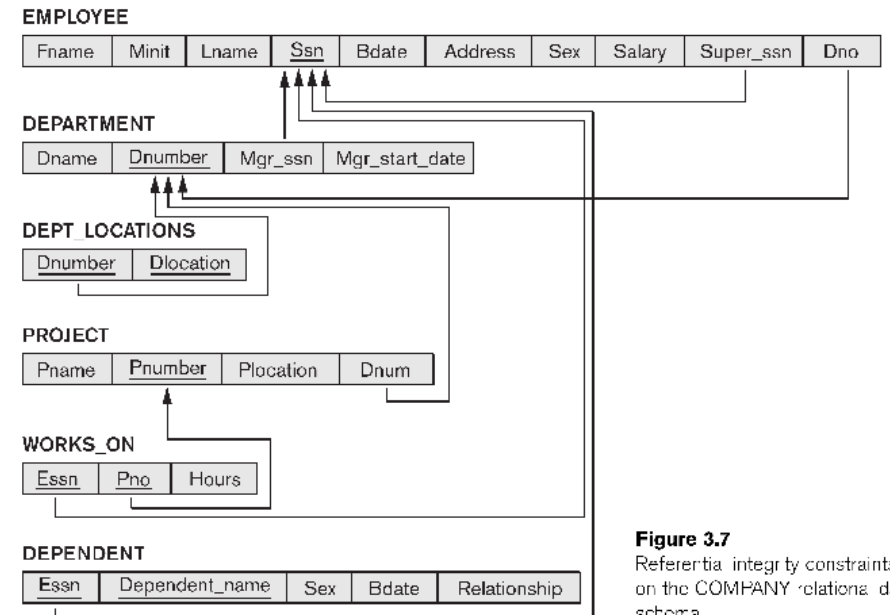


Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

Query 1

- Project number of all those projects in which smith is involved as manager

SELECT p.Pnumber

FROM PROJECT AS p INNER JOIN DEPARTMENT AS d ON
p.Dnum = d.Dnumber INNER JOIN EMPLOYEE AS e ON
d.Mgr_ssn = e.ssn

WHERE Lname = 'Smith'

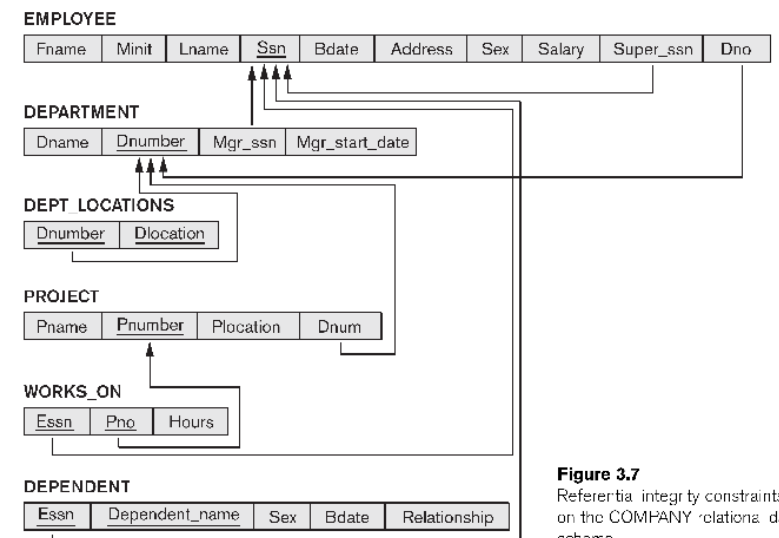


Figure 3.7
Referential integrity constraints displayed on the COMPANY relational database schema.

Query 1

```
SELECT DISTINCT Pname
```

```
FROM Project
```

```
Where Pnumber IN (Select wo.pno
```

```
From WORKS_ON AS wo INNER JOIN EMPLOYEE AS e ON e.ssn = wo.essn
```

```
WHERE Lname = 'smith';
```

```
)
```

```
OR Pnumber IN (SELECT p.Pnumber
```

```
FROM PROJECT AS p INNER JOIN DEPARTMENT AS d
```

```
ON p.Dnum = d.Dnumber INNER JOIN EMPLOYEE AS e ON d.Mgr_ssn = e.ssn
```

```
WHERE Lname = 'Smith');
```

Query 2

Select Name of all the employees who work the same (project, hours) combination on some project as 'John Smith' whose ssn is '123456789'

Step 1: find project and hours of ssn '123456789'

Step 2: find ssn of all employees with same working hours and project number

Step 3: find name of those employees.

Query 2

find project and hours of ssn '123456789'

```
SELECT Essn From WORKS_ON WHERE (Pno, Hours) IN  
(  
    SELECT Pno, Hours  
    FROM WORKS_ON  
    WHERE ESSN = '123456789');
```

Query 2

find ssn of all employees with same working hours and project number

```
SELECT Fname, Lname FROM EMPLOYEE WHERE ssn IN (
```

```
    SELECT Essn
```

```
    FROM WORKS_ON
```

```
    WHERE (Pno, Hours) IN
```

```
        (SELECT Pno, Hours
```

```
        FROM WORKS_ON
```

```
        WHERE ESSN = '123456789')
```

```
);
```

Query 2

Select Name of all the employees who work the same (project, hours) combination on some project as 'John Smith' whose ssn is '123456789'

```
SELECT Fname , Lname
```

```
FROM EMPLOYEE
```

```
WHERE ssn IN (
```

```
SELECT Essn
```

```
FROM WORKS_ON
```

```
WHERE (Pno, Hours) IN (SELECT Pno, Hours
```

```
FROM WORKS_ON
```

```
WHERE ESSN = '123456789'
```

```
));
```

Query 3

Show name of all those employees whose salary is greater than all employees of Dno = 5.

Step 1: salary of all employees of dno = 5.

Step 2: name of all employees with salary greater than all employees of dno = 5.

Query 3

Step 1: salary of all employees of dno = 5.

```
SELECT Fname , Lname FROM EMPLOYEE WHERE salary > ALL (  
    20000, 30000, 50000, 55000... );
```

Query 3

Step 2: name of all employees with salary greater than all employees of dno = 5.

```
SELECT Fname , Lname  
FROM Employee  
WHERE salary > ALL (SELECT salary  
FROM EMPLOYEE  
WHERE Dno = 5);
```